

The bounded-buffer problem :

In this problem, the producer and consumer processes share the following data structures:

```
int n;
semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0;
```

==

The readers-writers problem :

A database is to be shared among several concurrent processes. Some of these processes may want only to read, other may want to update the database.

The structure of a writer process

```
while (true) {
    wait (rw-mutex);
    /* writing is performed */
    signal (rw-mutex);
}

while (true) {
    while (true) {
        wait (mutex);
        read-count++;
        if (read-count == 1)
            wait (rw-mutex);
        signal (mutex);
        /* reading is performed */
        wait (mutex);
        read-count--;
        if (read-count == 0)
            signal (rw-mutex);
        signal (mutex);
    }
}
```

⇒ Reader-writer locks are most useful in the following situations:

1. In applications where it is easy to identify which process only read and which process only write shared data.

The Dining-Philosophers Problem:

→ The structure of philosopher i :

```
while (true) {  
    wait (chopstick[i]);  
    wait (chopstick[(i+1)%5]);
```

```
    /* eat for a while */
```

```
    signal (chopstick[i]);  
    signal (chopstick[(i+1)%5]);
```

```
    /* think for a while */
```

```
}
```

⇒ Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.

Several possible remedies to the deadlock problem are the following:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- Use an asymmetric solution - that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.

Problem in Semaphore:

- Handle manually.
- Not found what errors.
- Process check manually.
- Busy waiting (spinlocks)
- Deadlock

= Monitor: A monitor is a high-level synchronization construct that provides a convenient and effective mechanism for process synchronization.

→ This solution imposes the restriction that a philosopher may pick up her chopstick only if both of them are available.

Monitor solution to Dining Philosophers:-

monitor DiningPhilosophers

```
{  
    enum { THINKING, HUNGRY, EATING } state[5];
```

```
    condition self[5];
```

```
    void pickUp (int i) {
```

```
        state[i] = HUNGRY;
```

```
        test(i);
```

```
        if (state[i] != EATING) self[i].wait;
```

```
    }
```

```
    void putDown (int i) {
```

```
        state[i] = THINKING;
```

```
        test((i+4)%5);
```

```
        test((i+1)%5);
```

```
    }
```

```

void test (int i) {
    if (-(state[(i+4)%5] != EATING) && (state[i] = HUNGRY) &&
        (state[(i+1)%5] != EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}

```

```

initialization_code() {
    for (int i=0; i<5; i++) {
        state[i] = THINKING;
    }
}

```

⇒ Each philosopher i invokes the operations pickup(i) and putdown(i)

```

DiningPhilosophers.pickup(i);
    ** EAT **

```

```

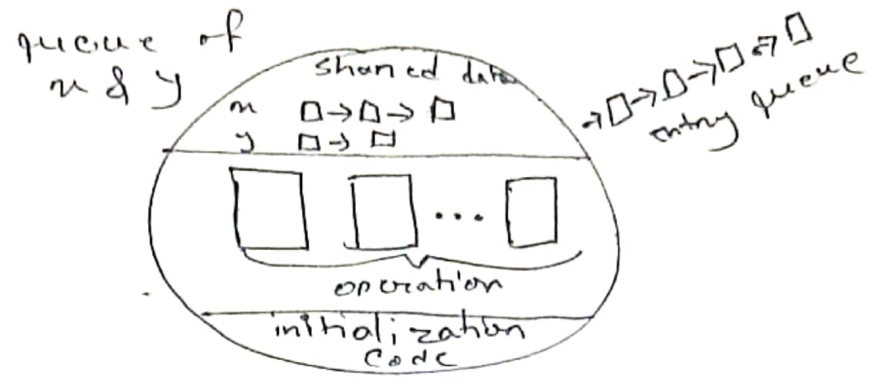
DiningPhilosophers.putdown(i);

```

□ No deadlock, but starvation is possible.

Solve producer & Consumer problem using Monitor.

Condition Construct:



— Again need to read 6.7